MOBILE APPLICATION SECURITY TESTING

Test Target: Android-Project.apk

MANAPPURAM OGL

SHIJIMOL S

NOVAVI STUDENT

01-04-2026

Contents

1.1 Executive Summary

In April 2026, a Mobile Application VAPT was conducted by Shijimol S on the Android application **Manappuram OGL (Android-Project.apk)** to evaluate its security posture and identify potential vulnerabilities affecting confidentiality, integrity, and availability.

The assessment included static analysis, reverse engineering, permission analysis, and limited dynamic testing. A total of findings was identified across multiple severity levels:

- Critical: 1
- High: 3
- Medium: 5
- Low: 4
- Informational: 3

The key issues include insecure network communication (cleartext traffic), application data backup enabled, exported services without proper access control, and improper permission usage. These vulnerabilities may lead to data exposure, unauthorized access, and increased attack surface.

Overall, the identified risks are primarily due to misconfigurations and insecure component exposure rather than complex exploitation techniques, indicating a moderate security risk.

1.2 Key Findings

The security assessment of the Manappuram OGL (Android-Project.apk) identified multiple vulnerabilities across different severity levels. The majority of issues are related to improper component exposure, insecure configurations, and excessive permission usage.

The most critical observations include:

- The application allows cleartext network communication (`usesCleartextTraffic="true"`), which exposes sensitive data to potential Man-in-the-Middle (MITM) attacks.
- Application data backup is enabled (`allowBackup="true"`), allowing attackers to extract sensitive data using ADB.
- Custom Firebase services are exported without proper access control, enabling unauthorized service invocation and potential push notification abuse.
- The application requests sensitive and high-risk permissions such as `READ_CONTACTS`, external storage access, and `WAKE_LOCK`, increasing the risk of data exposure and resource misuse.
- Insecure storage practices and FileProvider configuration may allow unauthorized access to application data stored on external storage.
- Some components are securely configured (e.g., non-exported services, receivers, and providers), indicating partial adherence to secure development practices.

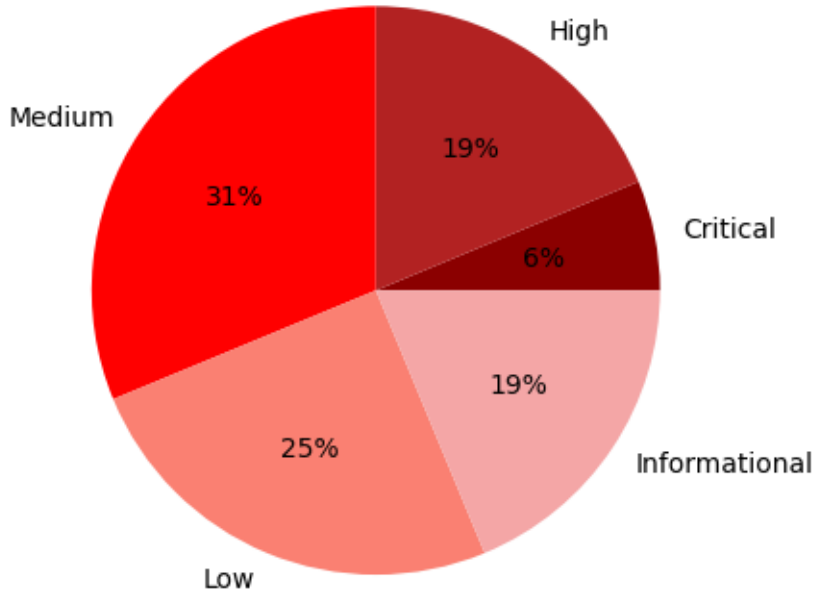
Overall, the findings indicate that the application has **moderate security risks**, primarily due to misconfigurations and improper access control rather than complex exploitation techniques.

1.2.1 Severity

Severity	# of Findings
Critical	1
High	3
Medium	5
Low	4
Informational	3

Table 1: Severity of Vulnerabilities

1.2.2 Vulnerability Severity Distribution



1.2.3 Issue Categorization

Issue Category	# of Findings
Insecure Communication	1
Data Storage & Data Leakage	3
Improper Component Exposure	4
Permission & Access Control Issues	3
Privacy Issues	2
Security Misconfiguration	1
Security Monitoring Weakness	1
Third-Party / Dependency Risk	1

Chapter 1

Vulnerability Details

1.1 Critical Findings

1.1.1 Insecure Network Communication

Severity: CRITICAL

Description

The application is configured with `android:usesCleartextTraffic="true"` in the AndroidManifest.xml file. This setting allows the application to transmit and receive data over unencrypted HTTP connections instead of secure HTTPS.

As a result, network communication between the mobile application and backend servers is not protected by encryption, making it susceptible to interception and tampering by attackers.

Discovery Method

- Static Analysis using JADX and MobSF
- Reviewed `AndroidManifest.xml`

Identified the following configuration:

```
android:usesCleartextTraffic="true"
```

Proof of Concept (PoC)

Step 1: Setup Proxy

- Configure device/emulator to route traffic through Burp Suite

Step 2: Launch Application

- Open the application and perform actions such as:
  - Login
  - API requests

Step 3: Intercept Traffic

- Observe intercepted requests in Burp Suite

Step 4: Verify Protocol

- Check if requests are sent over:

```
http://
```

Result:

- If HTTP traffic is observed → Cleartext communication confirmed

Impact Analysis

- Man-in-the-Middle (MITM) Attacks
- Sensitive Data Exposure
- Data Manipulation
- Session Hijacking

Mitigation

- Disable Cleartext Traffic
- Enforce HTTPS
- Implement Network Security Configuration
- Implement Certificate Pinning
- Validate Server Certificates

1.2 High Severity

1.2.1 Application Data Backup Enabled

Severity: HIGH

Description

The application is configured with `android:allowBackup="true"` in the `AndroidManifest.xml`. This setting allows the application's data to be backed up using Android Debug Bridge (ADB) or through the Android backup mechanism.

If enabled, attackers with physical access to the device or debugging access can extract sensitive application data without requiring root privileges (in some cases), leading to potential exposure of confidential information.

Discovery Method

- **Static Analysis** using JADX and MobSF
- Decompiled APK and reviewed `AndroidManifest.xml`

Identified Configuration:

```
android:allowBackup="true"
```

Proof of Concept (PoC)

Step 1: Connect Device / Emulator

```
adb devices
```

Step 2: Perform Backup

```
adb backup -f backup.ab com.manappuram.ogl
```

Step 3: Convert Backup File (Optional)

```
dd if=backup.ab bs=24 skip=1 | openssl zlib -d > backup.tar
```

Step 4: Extract Data

```
tar -xvf backup.tar
```

Result:

- Extracted application data may include:
  - SharedPreferences
  - Databases
  - Cached files
  - User/session data
- Sensitive data exposure confirmed

Impact Analysis

- Sensitive Data Exposure
- Unauthorized Data Access
- Reverse Engineering Support
- Privacy Violation

Mitigation

- Disable Backup
- Use Full Backup Content Control
- Encrypt Sensitive Data
- Avoid Storing Sensitive Data
- Implement Root/Debug Detection

1.2.2 Exported Custom Firebase Messaging Service Without Access Control

Severity: HIGH

Description

The application exposes a custom Firebase messaging service (`MyFirebasemessagingService`) with `android:exported="true"` in the `AndroidManifest.xml`. This configuration allows external applications to interact with the service without any permission restrictions.

Since this service handles Firebase Cloud Messaging (FCM) events, improper exposure may allow unauthorized entities to trigger the service or inject crafted intents, potentially leading to misuse of notification handling logic.

Discovery Method

- **Static Analysis** using JADX and MobSF
- Reviewed `AndroidManifest.xml`

Identified Configuration:

```
<service

android:name="com.manappuram.ogl.modules.notifications.MyFirebasemess
agingService"
    android:exported="true">
```

Proof of Concept (PoC)

Step 1: Connect Device / Emulator

```
adb devices
```

Step 2: Attempt to Start the Service

Using ADB:

```
adb shell am startservice \
-n
com.manappuram.ogl/.modules.notifications.MyFirebasemessagingService
```

Step 3: Observe Behavior

- If the service starts or logs activity → Externally accessible
- Monitor logs using:

```
adb logcat
```

Result:

- Service can be triggered externally → Unauthorized invocation confirmed

Impact Analysis

- Unauthorized Service Invocation
- Fake Notification Injection
- Abuse of Application Logic
- Increased Attack Surface

Mitigation

- Restrict Service Exposure
- Apply Permission Protection
- Validate Incoming Intents
- Limit Service Functionality
- Use Secure Messaging Handling

1.2.3 Exported Custom Firebase Instance ID Service Without Access Control

Severity: HIGH

Description

The application exposes a custom Firebase Instance ID service (`MyFirebaseInstanceIDService`) with `android:exported="true"` in the `AndroidManifest.xml`. This service is responsible for handling Firebase Instance ID events, including token generation and refresh.

Because the service is exported and lacks any permission protection, it can potentially be invoked by external applications. This creates a security risk where attackers may interfere with token handling mechanisms or trigger internal service logic without authorization.

Discovery Method

- **Static Analysis** using JADX and MobSF
- Decompiled APK and reviewed `AndroidManifest.xml`

Identified Configuration:

```
<service

android:name="com.manappuram.ogl.modules.notifications.MyFirebaseInst
```

```
anceIDService"
    android:exported="true">
```

Proof of Concept (PoC)

Step 1: Connect Device / Emulator

```
adb devices
```

Step 2: Attempt to Start the Service

Using ADB:

```
adb shell am startservice \
-n
com.manappuram.ogl/.modules.notifications.MyFirebaseInstanceIDService
```

Step 3: Monitor Logs

```
adb logcat
```

Result:

- If the service is triggered externally or logs are generated → Service is externally accessible

This confirms lack of access control

Impact Analysis

- Token Manipulation
- Push Notification Abuse
- Unauthorized Service Invocation
- Increased Attack Surface

Mitigation

- Restrict Service Exposure
- Apply Permission Protection
- Validate Caller Identity
- Secure Token Handling
- Follow Firebase Best Practices

1.3 Medium Findings

1.3.1 Excessive Permission Usage – READ_CONTACTS

Severity: MEDIUM

Description

The application requests the `android.permission.READ_CONTACTS` permission, which allows access to the user's contact list, including names, phone numbers, and email addresses.

During testing, no clear functional requirement was identified that justifies the need for accessing user contacts. Requesting such sensitive permissions without a valid use case violates the principle of least privilege and increases the risk of user data exposure.

Discovery Method

- **Static Analysis** using JADX and MobSF
- Decompiled APK and reviewed `AndroidManifest.xml`

Identified Configuration:

```
<uses-permission android:name="android.permission.READ_CONTACTS"/>
```

Proof of Concept (PoC)

Step 1: Install and Launch Application

- Install the APK on emulator/device

Step 2: Monitor Permission Requests

- Navigate through app features
- Observe that the app requests contact access

Step 3: Validate Usage

- No visible feature (e.g., contact sync, sharing) uses this permission

❖ Optional Verification:

Using MobSF:

- Check permission analysis report
- Confirm presence of `READ_CONTACTS`

Result:

- Permission is declared but not clearly justified → Over-permission confirmed

Impact Analysis

- Privacy Violation
- Data Leakage Risk
- Increased Attack Surface
- User Trust Impact

Mitigation

- Remove Unnecessary Permission
- Request Permission Only When Needed
- Follow Principle of Least Privilege
- Provide User Transparency
- Secure Data Handling

1.3.2 Insecure External Storage Usage

Severity: MEDIUM

Description

The application requests external storage permissions (`READ_EXTERNAL_STORAGE`, `WRITE_EXTERNAL_STORAGE`) and enables legacy storage behavior using `android:requestLegacyExternalStorage="true"`.

This configuration allows the application to store and access data on shared external storage, which is accessible by other applications on the device. External storage lacks proper access controls, making any sensitive data stored there vulnerable to unauthorized access.

Discovery Method

- **Static Analysis** using JADX and MobSF
- Reviewed `AndroidManifest.xml`

Identified Configuration:

```
<uses-permission
android:name="android.permission.READ_EXTERNAL_STORAGE"/>

<uses-permission
android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
```

```
android:requestLegacyExternalStorage="true"
```

Proof of Concept (PoC)

Step 1: Install and Run Application

- Install APK on device/emulator
- Perform actions that may store data (login, file download, etc.)

Step 2: Access External Storage

Using ADB:

```
adb shell
cd /sdcard/
ls
```

Step 3: Locate Application Data

- Navigate through directories to identify app-related files
- Example locations:
  - `/sdcard/Android/data/`
  - `/sdcard/Download/`

Step 4: Access Files

- Open or copy files without any restriction

Result:

- Application data stored in external storage is accessible → Insecure storage confirmed

Impact Analysis

- Sensitive Data Exposure
- Unauthorized Access by Other Apps
- Data Tampering
- Privacy Violation

Mitigation

- Avoid Using External Storage for Sensitive Data
- Disable Legacy Storage
- Use Scoped Storage (Modern Android)
- Encrypt Sensitive Files
- Minimize Permissions

1.3.3 Potential Sensitive Data Exposure via FileProvider Misconfiguration

Severity: MEDIUM

Description

The application defines a `FileProvider` with `android:grantUriPermissions="true"` in the `AndroidManifest.xml`. This configuration allows the application to grant temporary access to files via content URIs to other applications.

While `FileProvider` is a secure mechanism when properly configured, improper implementation—especially overly broad path definitions in `file_paths.xml`—can lead to unintended exposure of sensitive files.

If sensitive directories are exposed or URI permissions are granted without strict validation, malicious applications may gain access to internal or external files.

Discovery Method

- **Static Analysis** using JADX and MobSF
- Reviewed `AndroidManifest.xml`

Identified Configuration:

```
<provider
    android:name="androidx.core.content.FileProvider"
    android:exported="false"
    android:grantUriPermissions="true">
```

- Further analysis recommended on:

  `res/xml/file_paths.xml`

Proof of Concept (PoC)

Step 1: Identify Shared URIs

- Analyze code or runtime behavior to find file-sharing functionality

Step 2: Trigger File Sharing

- Perform actions like:
  - File upload
  - Share file feature

Step 3: Capture URI

Example:

```
content://com.manappuram.ogl.provider/...
```

Step 4: Attempt Unauthorized Access

Using ADB or a test app:

- Try accessing shared URI
- Attempt to read file content

Step 5: Check file_paths.xml

Look for risky configuration:

```
<external-path path="." />
```

Result:

- If broad paths are allowed → Sensitive file exposure possible

Impact Analysis

- Sensitive File Exposure
- Unauthorized Access
- Data Leakage
- Data Tampering

Mitigation

- Restrict File Paths
- Avoid Broad Path Definitions
- Validate URI Access
- Minimize File Exposure
- Use Read-Only Permissions Where Possible

1.3.4 Potential Misuse of WAKE_LOCK Permission

Severity: MEDIUM

Description

The application requests the `android.permission.WAKE_LOCK`, which allows it to keep the device awake (prevent CPU sleep and/or screen dimming) while running in the background.

While this permission is sometimes required for legitimate use cases (e.g., media playback, long-running tasks), improper or excessive use can allow the application to run continuously

in the background without user awareness. This behavior may lead to resource abuse and potential misuse for stealthy operations.

Discovery Method

- **Static Analysis** using JADX and MobSF
- Reviewed `AndroidManifest.xml`

Identified Configuration:

```
<uses-permission android:name="android.permission.WAKE_LOCK"/>
```

Proof of Concept PoC

Step 1: Install and Launch Application

- Install the APK on a device/emulator

Step 2: Monitor Background Behavior

- Use:

```
adb shell dumpsys power
```

- Check for active wake locks

Step 3: Observe CPU Usage

- Monitor device behavior:

```
adb shell top
```

Step 4: Keep App in Background

- Send app to background
- Observe if:
    - CPU remains active
    - Device does not enter sleep mode

Result:

- If wake lock is held continuously → Potential misuse confirmed

Impact Analysis

- Battery Drain
- Resource Abuse
- Stealth Activity
- Performance Degradation

Mitigation

- Use WAKE_LOCK Only When Necessary
- Implement Proper Wake Lock Handling
- Use Foreground Services (if required)
- Optimize Background Tasks
- Monitor Usage

1.3.5 Missing Explicit Exported Attribute in Activities

Severity: MEDIUM

Description

Multiple activities in the application do not explicitly define the `android:exported` attribute in the `AndroidManifest.xml`. The absence of this attribute can lead to ambiguous behavior depending on the Android version and configuration.

In older Android versions, activities with intent filters may be implicitly exported, making them accessible to external applications. If sensitive activities (such as authentication, payment, or account management screens) are unintentionally exposed, attackers may attempt to launch them directly and bypass normal application flow.

Discovery Method

- **Static Analysis** using JADX and MobSF
- Decompiled APK and reviewed `AndroidManifest.xml`

Observed Configuration:

- Multiple activities declared without:

```
android:exported="true/false"
```

Proof of Concept (PoC)

Step 1: Install and Launch Application

- Install APK on emulator/device

Step 2: Attempt to Launch Activity

Using ADB:

```
adb shell am start \
-n com.manappuram.ogl/.modules.pay.view.PaymentDetailActivity\
```

Step 3: Observe Behavior

- If activity opens without authentication → vulnerable
- If blocked → protected

Step 4: Repeat for Other Activities

- Test:
  - ResetMpin
  - ChangePasswordActivity

Result:

- Potential exposure depends on runtime behavior → Risk confirmed (conditional)

Impact Analysis

- Unauthorized Access to Sensitive Screens
- Authentication Bypass
- Business Logic Abuse
- Increased Attack Surface

Mitigation

- Explicitly Define Exported Attribute
- Restrict Sensitive Activities
- Avoid Unnecessary Intent Filters
- Implement Access Control
- Follow Android Best Practices

1.4 Low Findings

1.4.1 Exported Install Referrer Broadcast Receiver

Severity: LOW

Description

The application exposes a broadcast receiver (`AppMeasurementInstallReferrerReceiver`) with `android:exported="true"` to handle install referrer events (`com.android.vending.INSTALL_REFERRER`).

Although the receiver is protected by the system-level permission `android.permission.INSTALL_PACKAGES`, which restricts access to trusted sources such as the Google Play Store, the component remains externally exposed. This increases the application's attack surface and may introduce risks if permission enforcement is bypassed or misconfigured.

.

Discovery Method

- **Static Analysis** using JADX and MobSF
- Reviewed `AndroidManifest.xml`

Identified Configuration:

```
<receiver

android:name="com.google.android.gms.measurement.AppMeasurementInstal
lReferrerReceiver"
    android:permission="android.permission.INSTALL_PACKAGES"
    android:exported="true">
    <intent-filter>
        <action android:name="com.android.vending.INSTALL_REFERRER"/>
    </intent-filter>
</receiver>
```

Proof of Concept (PoC)

Step 1: Attempt to Send Broadcast

Using ADB:

```
adb shell am broadcast \
-a com.android.vending.INSTALL_REFERRER \
-n com.manappuram.ogl/.AppMeasurementInstallReferrerReceiver
```

Step 2: Observe Behavior

- If broadcast is blocked → permission enforced
- If accepted → potential misconfiguration

Result:

- In most cases, broadcast is restricted due to permission
  Confirms low-risk exposure but increased attack surface

Impact Analysis

- Increased Attack Surface
- Potential Broadcast Injection
- Data Integrity Risk
- Minimal Direct Impact

Mitigation

- Restrict Exposure
- Validate Incoming Data
- Use Updated APIs
- Monitor Permission Enforcement

1.4.2 Exported Firebase Broadcast Receivers with Permission Protection

Severity: LOW

Description

The application includes Firebase broadcast receivers (e.g., `FirebaseInstanceIdReceiver`) configured with `android:exported="true"` and protected by a signature-level permission (`com.google.android.c2dm.permission.SEND`).

These receivers are designed to handle Firebase Cloud Messaging (FCM) events such as message delivery. While the permission restriction ensures that only trusted system components (e.g., Google Play Services) can send broadcasts, the components remain externally exposed, increasing the application's attack surface.

If permission enforcement is bypassed (rare but possible in compromised environments), attackers may attempt to inject malicious broadcasts.

Discovery Method

- **Static Analysis** using JADX and MobSF
- Reviewed `AndroidManifest.xml`

Identified Configuration:

```
<receiver
    android:name="com.google.firebase.iid.FirebaseInstanceIdReceiver"
    android:permission="com.google.android.c2dm.permission.SEND"
    android:exported="true">
    <intent-filter>
        <action
android:name="com.google.android.c2dm.intent.RECEIVE"/>
    </intent-filter>
</receiver>
```

Proof of Concept (PoC)

Step 1: Attempt to Send Broadcast

Using ADB:

```
adb shell am broadcast \
-a com.google.android.c2dm.intent.RECEIVE \
-n com.manappuram.ogl/.FirebaseInstanceIdReceiver
```

Step 2: Observe Behavior

- Broadcast should be blocked due to permission restriction
- Monitor logs:

```
adb logcat
```

Result:

- Broadcast injection fails under normal conditions
  Confirms low-risk, permission-protected exposure

Impact Analysis

- Increased Attack Surface
- Broadcast Injection
- Notification/Data Manipulation Risk
- Minimal Practical Impact

Mitigation

- Minimize Exported Components
- Enforce Strict Permission Checks
- Validate Incoming Broadcast Data
- Keep Firebase SDK Updated

1.4.3 Access to Advertising ID (AD_ID) – Potential Privacy Risk

Severity: LOW

Description

The application requests access to the Advertising ID using the permission `com.google.android.gms.permission.AD_ID`. The Advertising ID is a unique, user-resettable identifier provided by Google Play Services, primarily used for advertising and analytics purposes.

While this permission is commonly used in mobile applications, improper usage—such as collecting, storing, or transmitting the Advertising ID without user consent—can lead to privacy concerns and potential misuse of user tracking data.

Discovery Method

- **Static Analysis** using JADX and MobSF
- Reviewed `AndroidManifest.xml`

Identified Configuration:

```
<uses-permission android:name="com.google.android.gms.permission.AD_ID"/>
```

Proof of Concept (PoC)

Step 1: Install and Run Application

- Launch the application on a device/emulator

Step 2: Monitor Network Traffic

Using Burp Suite:

- Intercept outgoing API requests
- Look for Advertising ID in request parameters

Step 3: Analyze Data Usage

- Check if AD_ID is:
  - Collected
  - Stored
  - Sent to backend servers

Result:

- Presence of AD_ID permission confirms tracking capability
- Further dynamic testing required to confirm actual usage

Impact Analysis

- User Tracking
- Privacy Concerns
- Data Sharing Risk
- Regulatory Compliance Issues

Mitigation

- Request User Consent
- Minimize Data Collection
- Follow Privacy Regulations
- Provide Opt-Out Mechanism
- Secure Data Transmission

1.4.4 Crash Reporting Disabled (Firebase Crashlytics Disabled)

Severity: LOW

Description

The application has disabled Firebase Crashlytics by setting `firebase_crashlytics_collection_enabled="false"` in the AndroidManifest.xml. Crashlytics is used to collect crash reports and runtime exceptions, which help developers monitor application stability and identify security-related issues.

Disabling crash reporting reduces visibility into application failures, including those that may be triggered by security flaws or exploitation attempts. This weakens the application's ability to detect and respond to potential vulnerabilities in a timely manner.

Discovery Method

- **Static Analysis** using JADX and MobSF
- Reviewed `AndroidManifest.xml`

Identified Configuration:

```
<meta-data
    android:name="firebase_crashlytics_collection_enabled"
    android:value="false"/>
```

Proof of Concept (PoC)

Step 1: Install and Run Application

- Launch the application on a device/emulator

Step 2: Trigger a Crash (Test Scenario)

- Perform invalid actions or force an exception (if possible)

Step 3: Monitor Logs

Using ADB:

```
adb logcat
```

Step 4: Verify Reporting

- Check if crash is logged remotely (Crashlytics dashboard)
- Observe that no reporting occurs

Result:

- Crash events are not collected → Monitoring disabled confirmed

Impact Analysis

- Delayed Vulnerability Detection
- Lack of Incident Visibility
- Reduced Debugging Capability
- Increased Risk of Undetected Exploits

Mitigation

- Enable Crashlytics
- Implement Secure Logging
- Monitor Crash Reports Regularly
- Avoid Logging Sensitive Data
- Use Additional Monitoring Tools

1.5 Informational Findings

1.5.1 Use of Firebase Services (Messaging, Analytics, Initialization)

Severity: INFORMATIONAL

Description

The application integrates multiple Firebase services, including Firebase Cloud Messaging (FCM), Firebase Analytics, and Firebase initialization components.

These services are used to enable features such as push notifications, user behavior tracking, and backend communication. While Firebase provides robust and secure infrastructure, its integration increases the application's overall attack surface.

Improper configuration or insecure implementation of Firebase services may lead to issues such as unauthorized message handling, data exposure, or misuse of analytics data.

Discovery Method

- **Static Analysis** using JADX and MobSF
- Reviewed `AndroidManifest.xml`

Identified Components:

- Firebase Messaging Services
- Firebase Instance ID Services
- Firebase Initialization Provider
- Firebase Analytics Components

Example:

```
com.google.firebase.messaging.FirebaseMessagingService
com.google.firebase.iid.FirebaseInstanceIdService
com.google.firebase.provider.FirebaseInitProvider
```

Proof of Concept (PoC)

Step 1: Install and Launch Application

- Run the application on emulator/device

Step 2: Monitor Network Traffic

Using Burp Suite:

- Intercept requests to Firebase endpoints
- Example domains:

```
firebase.googleapis.com
fcm.googleapis.com
```

Step 3: Observe Behavior

- Push notifications received
- Analytics events sent

Result:

- Firebase services actively used → Integration confirmed

Impact Analysis

- Increased Attack Surface
- Push Notification Risks
- Data Collection & Privacy Concerns
- Dependency Risk

Mitigation

- Follow Firebase Security Best Practices
- Validate Incoming Messages
- Secure Data Transmission
- Minimize Data Collection
- Implement Proper Access Control

1.5.2 Use of Google Analytics Components (User Behavior Tracking)

Severity: INFORMATIONAL

Description

The application integrates Google Analytics components to collect and analyze user behavior data. These components are used to track user interactions, session activity, and application usage patterns.

While analytics tools are commonly used to improve user experience and application performance, improper handling of analytics data may lead to privacy concerns. If user data is

collected, transmitted, or stored without proper consent or protection, it may expose sensitive behavioral information.

Discovery Method

- **Static Analysis** using JADX and MobSF
- Reviewed `AndroidManifest.xml`

Identified Components:

- Analytics services
- Analytics receivers
- Job services

Example:

```
com.google.android.gms.analytics.AnalyticsService
com.google.android.gms.analytics.AnalyticsReceiver
com.google.android.gms.analytics.AnalyticsJobService
```

Proof of Concept (PoC)

Step 1: Install and Launch Application

- Run the application on emulator/device

Step 2: Monitor Network Traffic

Using Burp Suite:

- Intercept outgoing requests
- Look for analytics-related endpoints

Step 3: Observe Data Transmission

- Identify analytics payloads such as:
  - User interactions
  - Event tracking data

Result:

- Analytics data transmission observed → Tracking functionality confirmed

Impact Analysis

- User Behavior Tracking
- Privacy Concerns
- Data Sharing Risk
- Regulatory Compliance Risk

Mitigation

- Obtain User Consent
- Minimize Data Collection
- Anonymize Data
- Follow Privacy Regulations
- Provide Opt-Out Mechanism

1.5.3 Use of Third-Party Libraries (ZXing, EasyPermissions)

Severity: INFORMATIONAL

Description

The application utilizes third-party libraries, including ZXing for barcode/QR code scanning and EasyPermissions for runtime permission handling.

While third-party libraries accelerate development and provide reusable functionality, they may introduce security risks if not properly maintained. Vulnerabilities in these libraries, outdated versions, or insecure implementations can affect the overall security of the application.

Discovery Method

- **Static Analysis** using JADX and MobSF
- Decompiled APK and reviewed package structure and dependencies

Identified Libraries:

- ZXing (barcode scanning functionality)
- EasyPermissions (permission management)

Example components:

```
com.journeyapps.barcodescanner.CaptureActivity
pub.devrel.easypermissions.AppSettingsDialogHolderActivity
```

Proof of Concept (PoC)

Step 1: Analyze Decompiled Code

- Open APK in JADX
- Search for library packages:

```
com.journeyapps.barcodescanner
pub.devrel.easypermissions
```

Step 2: Identify Library Usage

- Locate activities and classes related to these libraries
- Confirm integration in application workflow

Step 3: Check Version (if possible)

- Review Gradle files or embedded metadata
- Compare with latest versions (manual check)

Result:

- Third-party libraries identified → Dependency usage confirmed

Impact Analysis

- Supply Chain Risk
- Outdated Library Risk
- Expanded Attack Surface
- Indirect Vulnerabilities

Mitigation

- Keep Libraries Updated
- Monitor Vulnerabilities
- Use Trusted Sources
- Minimize Dependencies
- Perform Dependency Scanning

Chapter 2

Conclusion

The mobile application security assessment of **Manappuram OGL (Android-Project.apk)** revealed multiple security issues across different severity levels, highlighting several areas that require improvement to enhance the overall security posture of the application.

The most critical concern identified is the use of cleartext communication, which exposes sensitive data to potential interception through Man-in-the-Middle (MITM) attacks. In addition, high severity issues such as application data backup being enabled and exported services without proper access control significantly increase the risk of unauthorized data access and misuse of application components.

Medium severity findings indicate weaknesses in permission management, insecure data storage practices, and improper component configurations, which contribute to an expanded attack surface and increase the likelihood of data leakage or unauthorized access. Furthermore, low severity and informational findings highlight concerns related to privacy, monitoring gaps, third-party dependencies, and analytics usage, which, although not directly exploitable, may impact user trust and regulatory compliance if not properly managed.

Overall, the identified vulnerabilities are primarily the result of misconfigurations and insecure implementation practices rather than complex exploitation techniques. This indicates that the application can be significantly strengthened by adhering to secure coding standards, implementing proper access controls, minimizing permissions, and enforcing secure communication mechanisms.

By addressing the identified issues—especially those related to network security, component exposure, and data protection—the organization can greatly reduce potential security risks and improve the resilience of the application against real-world attacks. Continuous security testing, regular updates, and adherence to industry best practices are strongly recommended to maintain a robust security posture.